# FinGPT Part 2 Development Guide

This document summarizes the current repository as it exists today and is intended to help development work move quickly without re-discovering the architecture.

## Repo Purpose

This project implements a local, inference-only financial news pipeline:

1. `Agent 1` extracts a structured `NewsFingerprint` from article text.
2. `Agent 2` converts that fingerprint into a `TradingSignal`.
3. The `backtest` package runs the same pipeline over a labeled dataset and evaluates realized returns.

The core design choice is that decisions are derived from real token log-probabilities read from vLLM, not from self-reported numbers produced by the model.

## High-Level Architecture

### Main flow

- `pipeline.py`
  - Live entry point.
  - Fetches articles from Alpaca or Finnhub.
  - Concatenates `headline + summary`.
  - Runs `extract_fingerprint()` then `generate_signal()`.
  - Saves results to `output/signals_<timestamp>.json`.

### Agent 1

- agent1/extractor.py
  - Loads or reuses a local vLLM engine.
  - Uses guided decoding for structured fact extraction.
  - Uses a second logits-based path for sentiment classification.
- agent1/schema.py
  - Defines `NewsFingerprint`.
- agent1/prompt.py
  - Extraction prompt and sentiment CoT/scoring prompt pieces.

### Agent 2

- agent2/reasoner.py
  - Reuses Agent 1's vLLM engine when `SHARE_SINGLE_LLM_BETWEEN_AGENTS=true`.
  - Runs strategy CoT generation and then logits-based A/B/C scoring.
  - Applies PMI correction using cached null-context logprobs.
- agent2/schema.py
  - Defines `TradingSignal`.
- agent2/prompt.py

    - Strategy CoT prompt, decision prefix, and score tokens.

### Shared logits helper

- vllm_logits_client.py
    - Encapsulates the two-phase vLLM pattern:
        - Phase 1: generate `<think>...</think>` reasoning.
        - Phase 2: append a deterministic decision prefix and read prompt token logprobs.
    - Supports both single-item and batched scoring.
    - Contains a legacy self-reported-logits path kept mostly for reference, not the main pipeline.

### Backtest

- backtest/backtester.py
    - Batch orchestration for end-to-end evaluation.
    - Calls Agent 1 in batch, then Agent 2 in batch, then fetches realized returns.
- backtest/dataset_parser.py
    - Loads `.csv`, `.parquet`, or Hugging Face datasets.
    - Normalizes columns to `input`, `output`, `answer`, `ticker`.
    - Extracts article text and date ranges from FinGPT-style prompts.
- backtest/price_fetcher.py
    - Fetches daily data from `yfinance`.
    - Computes close-to-close realized returns.
    - Maintains both in-memory and disk caches.
- backtest/run_backtest.py
    - CLI wrapper for running backtests and printing metrics.

### Ingestion

- ingestion/news_fetcher.py
    - Supports `alpaca` and `finnhub`.
    - Finnhub path includes retry, pacing, and deduplication.

## Data Contracts

#### `NewsFingerprint`

Fields defined in `agent1/schema.py`:

- Fact extraction fields:
    - `source`
    - `published_at`
    - `headline`
    - `companies_named`
    - `event_keywords`
- Sentiment fields:
    - `sentiment_label`
    - `sentiment_score`
    - `sentiment_confidence`

    - `sentiment_probabilities`
    - `sentiment_logits`
    - `calibration_T`
- Pass-through context:
    - `article_text`

Important validation behavior:

- `companies_named` must be non-empty or the fingerprint is rejected.
- `event_keywords` are normalized to lowercase.

### `TradingSignal`

Fields defined in `agent2/schema.py`:

- `ticker`
- `direction`: `long | short | neutral`
- `strategy_tag`: one of `momentum | mean_reversion | event_driven | macro | none`
- `confidence`
- `cot`
- Optional audit fields:
    - `signal_logits`
    - `signal_probabilities`
    - `calibration_T`

## How Inference Actually Works

### Agent 1 sentiment path

Agent 1 does not ask the model to emit a final sentiment score directly.

It performs:

1. Structured fact extraction with guided decoding.
2. A sentiment chain-of-thought prompt that stops at `</think>`.
3. A logits scoring step using `SENTIMENT_DECISION_PREFIX = "Sentiment: "` and the class list:
    - `POSITIVE`
    - `NEGATIVE`
    - `NEUTRAL`
4. Python-side post-processing:
    - `softmax(logits / CALIBRATION_T)`
    - argmax for label
    - max probability for confidence

If logits parsing fails, the code falls back to a uniform distribution and marks the article `NEUTRAL` rather than crashing.

### Agent 2 signal path

Agent 2 follows the same two-phase pattern but scores single-letter tokens instead of full action words:

- Score tokens: `["A", "B", "C"]`
- Strategy mapping:
  - `A -> BUY -> long`
  - `B -> HOLD -> neutral`
  - `C -> SELL -> short`

This avoids tokenization and prior-bias issues from directly scoring `BUY/HOLD/SELL`.

Agent 2 then applies PMI correction:

- `pmi_logits = raw_logits - null_logprobs`

The null-context prior is:

- Cached in memory for the active process.
- Persisted to `PMI_PRIOR_PATH` on disk.
- Invalidated automatically when model path, decision prefix, or score tokens change.

If Agent 2 cannot parse the scoring result, it returns `None` in strict mode and writes a diagnostic file.

## Batch Behavior

The repo is optimized around batching.

### Agent 1 batch

`extract_fingerprint_batch(article_texts)` performs:

1. One batched guided-decoding extraction call.
2. One batched CoT generation call for sentiment.
3. One batched scoring call for sentiment choices.

### Agent 2 batch

`generate_signal_batch(fingerprints)` performs:

1. One batched CoT generation call.
2. One batched scoring call over A/B/C choices.

### Backtest batch size

- `_BATCH_SIZE = 10` in `backtest/backtester.py`.

For each batch the practical flow is:

1. Agent 1 extraction.
2. Agent 1 sentiment scoring.
3. Agent 2 strategy scoring for valid fingerprints only.
4. Price fetch and metric assembly per row.

## Configuration and Environment

Primary config lives in config.py and `.env`.

### Important environment variables

- `FINGPT_MODEL_PATH`
  - Required for the local model and tokenizer.
- `FINGPT_ADAPTER_PATH`
  - Present in config/example, but not actively wired into the vLLM load path in the current code.
- `SHARE_SINGLE_LLM_BETWEEN_AGENTS`
  - Reuse one vLLM engine across both agents.
- `FINGPT_CALIBRATION_T`
  - Temperature for post-softmax calibration.
- `FINGPT_LOGITS_MAX_TOKENS`
  - CoT token budget for logits inference.
- `FINGPT_PMI_PRIOR_PATH`
  - Disk path for Agent 2 null-context prior cache.
- `FINGPT_YF_CACHE_PATH`
  - Disk cache for realized returns from `yfinance`.
- `FINGPT_DIAG_MD_DIR`
  - Optional base directory for markdown debug outputs.
- `NEWS_PROVIDER`
  - `alpaca` or `finnhub`.
- `ALPACA_API_KEY`, `ALPACA_API_SECRET`
- `FINNHUB_API_KEY`
- `FINNHUB_TIMEOUT_SEC`
- `FINNHUB_MAX_CALLS_PER_SEC`
- `FINNHUB_MAX_RETRIES`
- `FINNHUB_RETRY_BASE_DELAY_SEC`

### Notable config mismatch

`config.py` still defines Anthropic-related constants:

- `ANTHROPIC_API_KEY`
- `CLAUDE_MODEL`
- `CLAUDE_MAX_TOKENS`
- `CLAUDE_THINKING_BUDGET`

But the active Agent 2 implementation in `agent2/reasoner.py` is local vLLM-based, not Anthropic-based. Those constants currently look like leftovers from an earlier design and should not be treated as part of the main runtime path.

## Runtime Outputs

### Live pipeline output

- JSON file in `output/signals_<timestamp>.json`
- Each record is a serialized `TradingSignal`

### Backtest output

- CSV file, default `output/backtest_results.csv`
- Row-level fields include:
  - dataset metadata
  - Agent 1 sentiment results
  - Agent 2 signal results
  - realized return
  - position
  - strategy return
  - `skipped_reason`

### Common skip reasons

- `fingerprint_failed`
- `signal_failed`
- `price_fetch_failed`

### Diagnostics and logs

- Agent 1 markdown debug outputs:
  - `<diag_dir>/agent1`
- Agent 2 markdown debug outputs:
  - `<diag_dir>/agent2`
- Agent 2 failure diagnostics:
  - `<diag_dir>/agent2_failures`
- Agent 2 CoT logs:
  - `logs/cot_<ticker>_<timestamp>.txt`

## Backtest Metrics

`compute_metrics()` returns:

- `total_rows`
- `successful_rows`
- `skip_rate`
- `direction_accuracy`
- `long_accuracy`
- `short_accuracy`
- `mean_strategy_return`
- `std_strategy_return`
- `annualized_sharpe`
- `total_pnl`
- `vs_fingpt_accuracy`

Direction labels are compared against realized movement computed by `direction_from_return()`, which uses a default neutral threshold of `0.001`.

## Dependencies

Dependencies are listed in `requirements.txt`. The main runtime groups are:

- Model/runtime:
  - `transformers`
  - `torch`
  - `peft`
  - vLLM is required by the code but is not listed in `requirements.txt`; it must be installed separately in the execution environment.
- Data and evaluation:
  - `pandas`
  - `pyarrow`
  - `datasets`
  - `yfinance`
- Infra:
  - `python-dotenv`
  - `requests`
  - `pydantic`
- Dev:
  - `pytest`
  - `notebook`

## Entry Points Developers Will Actually Use

### Run the live pipeline

```
python pipeline.py AAPL MSFT NVDA
```

### Run a full backtest

```
python -m backtest.run_backtest --dataset "FinGPT/fingpt-forecaster-dow30-202305-202405" --metrics
```

### Run a smaller smoke-test backtest

```
python -m backtest.run_backtest --dataset "FinGPT/fingpt-forecaster-dow30-202305-202405" --max-rows 50 --metrics
```

### Use a custom local dataset and output path

```
python -m backtest.run_backtest --dataset "your_data.csv" --output "output/my_backtest.csv" --metrics
```

# Where To Change Things

## Change extraction behavior

- Prompting: `agent1/prompt.py`
- Parsing/validation: `agent1/extractor.py`, `agent1/schema.py`

## Change sentiment classes or calibration

- Class order: `config.py`
- Decision prefix / scoring prompt: `agent1/prompt.py`
- Softmax behavior: `vllm_logits_client.py` and `agent1/extractor.py`

## Change trading decision behavior

- Strategy prompt and score tokens: `agent2/prompt.py`
- Strategy mapping and PMI logic: `agent2/reasoner.py`
- Signal schema: `agent2/schema.py`

## Change batch size or result assembly

- `backtest/backtester.py`

## Change market data behavior

- `backtest/price_fetcher.py`

## Change live news provider behavior

- `ingestion/news_fetcher.py`

# Current Caveats and Development Notes

## 1. Tests are out of sync with the implementation

The tests under tests/test_extractor.py and tests/test_reasoner.py still target an older architecture:

- They refer to `_model`, `_tokenizer`, and Anthropic client behavior.
- The current code uses vLLM engines plus prompt-logprob scoring.
- Schema expectations in tests also include fields like `figures_quoted` that no longer exist in the current `NewsFingerprint`.

Before relying on CI confidence, these tests should be rewritten against the current vLLM-based pipeline.

## 2. README and comments may still reflect transitional design history

Some repo text still references older assumptions or intermediate states. When in doubt, treat the runtime code in:

- `agent1/extractor.py`
- `agent2/reasoner.py`
- `vllm_logits_client.py`

- `backtest/backtester.py`

as the source of truth.

### 3. `FINGPT_ADAPTER_PATH` is not currently used in model loading

It is exposed in config and `.env.example`, but the current `_load_model()` implementations do not apply a LoRA adapter path.

### 4. vLLM is a hard runtime dependency

The main code imports vLLM dynamically, but the environment must provide it. This is especially important when recreating the project outside the notebook environment.

## Recommended Development Workflow

1. Start from `config.py`, `.env.example`, and the relevant entry point.
2. For inference changes, trace:
    - prompt file
    - schema file
    - extractor/reasoner
    - `vllm_logits_client.py`
3. For evaluation changes, trace:
    - `dataset_parser.py`
    - `backtester.py`
    - `price_fetcher.py`
4. Treat tests as needing modernization before using them as a safety net.
5. Use the notebooks mainly for experiments and reruns, not as the primary source of architecture truth.